

MODULE 2

Functional and non-functional requirements

- **Software system requirements** are mainly classified into:
 - Functional requirements
 - Non-functional requirements

Functional Requirements

- Describe **what the system should do**.
- Specify the **services or functions** the system must provide.
- Define how the system should:
 - React to particular inputs.
 - Behave in different situations.
 - Perform specific tasks.
- Depend on:
 - Type of software being developed.
 - Expected users.
 - Organization needs.
- User-level functional requirements are usually high-level descriptions.
- System-level functional requirements provide detailed descriptions of services.
- Functional requirements should be:
 - **Complete** – include all required services.
 - **Consistent** – should not contain conflicting or contradictory requirements.

Examples (Mentcare System):

- A user shall be able to search appointment lists for all clinics.
- The system shall generate a daily list of patients expected for appointments in each clinic.

- Each staff member shall be uniquely identified by an **8-digit employee number**.

Non-functional Requirements

- Describe **constraints or quality attributes** of the system rather than specific functions.
- Define properties such as:
 - Performance
 - Reliability
 - Security
 - Availability
 - Usability
 - Response time
- Usually apply to the **entire system** rather than individual features.
- Often more critical than functional requirements because failure to meet them can make the system unusable.
- May influence the overall system architecture.
- A single non-functional requirement (such as security) can lead to additional functional requirements.

Types of Non-functional Requirements

- **Product Requirements**
 - Specify how the software should behave or perform.
 - Examples:
 - Execution speed
 - Reliability
 - Security
 - Usability
 - Memory usage
 - Example:

- The Mentcare system shall be available during working hours (Mon–Fri, 08:30–17:30).
- Downtime during working hours shall not exceed **5 seconds per day**.
- **Organizational Requirements**
 - Derived from organization policies and procedures.
 - Include development standards, operational procedures, or implementation constraints.
 - Example:
 - Users must authenticate using their health authority identity card.
- **External Requirements**
 - Arise from factors outside the system and development process.
 - Include legal, regulatory, ethical, or interoperability requirements.
 - Example:
 - The system shall implement patient privacy provisions specified by applicable standards.

Difference between Functional and Non-functional Requirements

- **Functional Requirements**
 - Describe **what the system does**.
 - Focus on services and behaviour.
 - Example: Generate daily patient appointment lists.
 - **Non-functional Requirements**
 - Describe **how well the system performs or the constraints under which it operates**.
 - Focus on quality attributes and system-wide properties.
 - Example: System downtime should not exceed 5 seconds per day.
-

Requirements Engineering Processes

- **Requirements Engineering (RE)** is the process of identifying, analyzing, documenting, and checking the services and constraints required for a software system.
- The RE process is generally **iterative**, meaning its activities may be repeated and overlap until the requirements are finalized.
- The final output is usually a **System Requirements Document**.

Main Activities of the Requirements Engineering Process

- **Feasibility Study**
 - Determines whether the proposed system is technically and financially feasible.
 - Evaluates if the system will be useful for the business.
 - Helps management decide whether to proceed with development.
- **Requirements Elicitation and Analysis**
 - Collects requirements from customers, users, and stakeholders.
 - Studies the application domain and system constraints.
 - Discovers required services, performance needs, and hardware/software limitations.
 - Analyzes and resolves conflicts between different stakeholder requirements.
- **Requirements Specification**
 - Converts gathered requirements into a clear and organized document.
 - Produces user requirements and system requirements in a standard format.
 - Serves as a reference for developers and customers.
- **Requirements Validation**
 - Checks whether the documented requirements accurately represent customer needs.
 - Ensures requirements are:
 - Complete
 - Consistent
 - Correct

- Realistic
 - Verifiable
- Identifies errors, omissions, and conflicts before development begins.
- **Requirements Management (Requirements Change)**
 - Handles changes to requirements during system development.
 - Tracks, evaluates, and updates requirements as business needs evolve.
 - Ensures modifications are properly documented and controlled.

Characteristics of the RE Process

- Activities are **interleaved and iterative**, not strictly sequential.
 - Different projects may spend different amounts of time on each activity.
 - Requirements are gradually refined through multiple iterations.
 - The process may stop after gathering all user requirements or continue until detailed system requirements are completed.
 - Produces a well-defined **system requirements document** that guides software development.
-

Requirements Elicitation

- **Requirements elicitation** is the process of discovering, collecting, and understanding the requirements of a software system.
- It is carried out after the initial feasibility study.
- Software engineers work with customers, users, and stakeholders to understand:
 - The application domain.
 - Services the system should provide.
 - Performance requirements.
 - Hardware and software constraints.
 - Business needs and expectations.

Main Activities

- **Requirements Discovery and Understanding**

- Interact with stakeholders to gather their requirements.
- Identify domain requirements from users and existing documents.
- Understand what the stakeholders expect from the system.
- **Requirements Classification and Organization**
 - Organize the collected requirements into related groups.
 - Group similar requirements into coherent categories.
 - Often based on the system architecture or subsystems.
- **Requirements Prioritization and Negotiation**
 - Prioritize requirements based on importance and business value.
 - Identify conflicts between stakeholder needs.
 - Resolve disagreements through discussion and negotiation to reach a compromise.
- **Requirements Documentation**
 - Record the finalized requirements.
 - Prepare formal or informal requirements documents.
 - Feed the documented requirements into the next iteration of the requirements engineering process.

Difficulties in Requirements Elicitation

- Stakeholders may not know exactly what they want and may only express general ideas.
- They may find it difficult to clearly explain their needs or may request unrealistic features.
- Stakeholders often describe requirements using domain-specific language that requirements engineers may not fully understand.
- Different stakeholders may have conflicting requirements or express similar needs differently.
- Political factors within an organization may influence requirements, with managers requesting features that increase their control or influence.
- The business and economic environment can change during development, causing requirements to change over time.

Importance

- Helps identify the actual needs of customers and users.
 - Forms the foundation for requirements specification and system design.
 - Reduces misunderstandings and conflicts early in the software development process.
 - Ensures that the developed system meets stakeholder expectations.
-

Requirements Specification

- **Requirements Specification** is the process of writing down the **user requirements** and **system requirements** in a requirements document.
- It converts gathered requirements into a clear and structured form for development.
- System requirements are expanded versions of user requirements and provide more detail about how the required services should be delivered.
- The requirements document may form part of the development contract, so it should be as complete and accurate as possible.

Key Points

- User requirements should be easy for customers and end users to understand.
- System requirements are more detailed and may contain technical information.
- The software requirements document (SRS) is an official statement of what the developers must implement.
- As far as possible, the document should describe **what the system should do**, not **how it should do it**.

Requirements and Design

- Ideally:
 - Requirements describe **what** the system should do.
 - Design describes **how** the system will do it.
- In practice, requirements and design are closely related because:
 - An initial architecture may be needed to organize requirements.

- The system may need to interact with existing systems.
- Certain architectures may be required to satisfy non-functional or regulatory requirements.

Ways of Writing System Requirements

- **Natural Language**
 - Requirements are written as simple numbered sentences.
 - Easy for users and customers to understand.
 - Each sentence should express one requirement.
- **Structured Natural Language**
 - Uses a standard template or form.
 - Makes requirements more organized and reduces ambiguity.
- **Design Description Language**
 - Uses a language similar to programming languages but at a higher level.
 - Useful for interface specifications but rarely used.
- **Graphical Notations**
 - Uses diagrams along with text.
 - Common examples include UML use case diagrams and sequence diagrams.
- **Mathematical Specifications**
 - Uses mathematical concepts or formal logic.
 - Provides precise and unambiguous specifications but is difficult for most customers to understand.

Guidelines for Writing Requirements

- Use a standard format for all requirements.
- Use consistent language:
 - **“Shall”** for mandatory requirements.
 - **“Should”** for desirable requirements.
- Highlight important parts of the requirement.
- Avoid unnecessary computer jargon.

- Include the reason (rationale) for a requirement when needed.

Problems with Natural Language Specifications

- **Lack of clarity** – precise wording can be difficult.
- **Requirements confusion** – functional and non-functional requirements may get mixed together.
- **Requirements amalgamation** – multiple requirements may be combined into one statement, causing ambiguity.

Form-Based Specification Fields

- Definition of the function or entity.
 - Description of inputs and their source.
 - Description of outputs and their destination.
 - Information required for computation or related entities.
 - Description of the action to be performed.
 - Pre-conditions and post-conditions (if applicable).
 - Side effects of the operation (if any).
-

Requirements Validation

- **Requirements validation** is the process of checking whether the documented requirements define the **system that the customer actually wants**.
- It helps detect errors, omissions, inconsistencies, and unrealistic requirements before development.
- Validation is important because fixing requirement errors after delivery can cost **much more** than fixing implementation errors.

Requirements Checking

- **Validity Check**
 - Ensures the requirements support the customer's actual needs.
 - Verifies that the system provides the required functions.
- **Consistency Check**

- Ensures there are no conflicting or contradictory requirements.
- Different parts of the requirements document should agree with each other.
- **Completeness Check**
 - Confirms that all required functions and constraints are included.
 - No important requirement should be missing.
- **Realism Check**
 - Verifies that the requirements can be implemented using available technology, budget, and resources.
- **Verifiability Check**
 - Ensures each requirement can be tested.
 - It should be possible to create test cases to prove that the requirement has been satisfied.

Requirements Validation Techniques

- **Requirements Reviews**
 - A team systematically examines the requirements document for errors and inconsistencies.
 - Reviews may be formal or informal.
 - Both customers and developers should participate.
- **Prototyping**
 - An executable model or prototype is shown to users and customers.
 - Users interact with it to verify whether it meets their actual needs.
- **Test-Case Generation**
 - Test cases are developed from the requirements.
 - Creating tests often reveals ambiguous, incomplete, or incorrect requirements.

Review Checks

- **Verifiability** – Can the requirement be realistically tested?
- **Comprehensibility** – Is the requirement easy to understand?

- **Traceability** – Is the source or origin of the requirement clearly identified?
 - **Adaptability** – Can the requirement be changed without causing major effects on other requirements?
 - Requirements validation improves software quality by ensuring the requirements are **valid, consistent, complete, realistic, and verifiable** before implementation begins.
-

Requirements Change

- **Requirements change** refers to modifications made to software requirements during or after development.
- Changes are common because business needs, technology, and user expectations evolve over time.
- **Requirements management** is the process of managing and controlling these changing requirements throughout requirements engineering and system development.

Why Requirements Change

- **Business and technical environments change**
 - New hardware or software may be introduced.
 - The system may need to interface with other systems.
 - Business priorities may change.
 - New laws or regulations may require system updates.
- **Customers and end users have different needs**
 - The people paying for the system are often different from those using it.
 - Organizational or budget constraints may conflict with user expectations.
 - After deployment, new features may be needed to satisfy users.
- **Large systems have diverse users**
 - Different user groups may have conflicting requirements and priorities.
 - The final requirements are often a compromise and may need revision after practical use.

Requirements Management

- Manages changing requirements during development and after deployment.
- Keeps track of individual requirements and their relationships.
- Assesses the impact of proposed changes before implementation.
- Establishes a formal process for proposing, evaluating, and recording changes.

Requirements Management Planning

- **Requirements Identification**
 - Each requirement should have a unique identifier.
 - Enables cross-referencing and traceability.
- **Change Management Process**
 - Defines how requirement changes are proposed, evaluated, and approved.
 - Assesses the cost and impact of changes.
- **Traceability Policies**
 - Define relationships between requirements and between requirements and system design.
 - Help analyze the effects of modifications.
- **Tool Support**
 - Specialized tools, databases, or spreadsheets may be used to store and manage requirements and changes.

Requirements Change Management Process

- **Problem Analysis and Change Specification**
 - A requirement problem or change request is identified.
 - The proposal is analyzed to check its validity.
 - The requester may refine or withdraw the change based on the analysis.
- **Change Analysis and Costing**
 - The impact of the proposed change is evaluated.
 - Traceability information and knowledge of the existing system are used to estimate cost and consequences.

- A decision is made on whether to accept the change.
 - **Change Implementation**
 - The requirements document is updated.
 - If necessary, the system design and implementation are also modified.
 - The requirements document should be organized so changes can be made easily without extensive rewriting.
-

System Modeling

- **System modeling** is the process of developing **abstract models** of a system, with each model presenting a different view or perspective of that system.
- It usually represents the system using **graphical notations**, most commonly based on the **Unified Modeling Language (UML)**.
- System models help analysts understand system functionality and communicate ideas with customers and developers.
- Models are used during:
 - Requirements engineering to derive requirements.
 - System design to describe the system to developers.
 - After implementation to document the system's structure and operation.

Benefits of System Modeling

- Simplifies understanding of complex systems.
- Eases project management tasks.
- Provides complete views of the system as well as detailed views of subsystems.
- Clarifies structures and relationships between components.
- Offers a communication framework for ideas within and between teams.
- Helps generate new ideas and design possibilities.
- Supports quality assurance and testing.
- Is generally platform independent.

Existing and Planned System Models

- **Models of existing systems**
 - Help understand current functionality.
 - Identify strengths and weaknesses.
 - Serve as a basis for defining requirements for a new system.
- **Models of new systems**
 - Help explain proposed requirements to stakeholders.
 - Support discussion of design proposals.
 - Document the system for implementation.
 - In model-driven engineering, they may be used to generate complete or partial implementations.

System Perspectives (Types of Models)

- **External Perspective**
 - Models the system's context or environment.
 - Represented using **context models**.
- **Interaction Perspective**
 - Models interactions between the system and its environment or internal components.
 - Represented using **use case diagrams** and **sequence diagrams**.
- **Structural Perspective**
 - Models the organization of the system and relationships between its components.
 - Represented using **class diagrams**.
- **Behavioral Perspective**
 - Models the dynamic behavior of the system and its response to events.
 - Represented using **activity diagrams** and **state diagrams**.

UML Diagram Types

- **Activity Diagram**
 - Shows activities involved in a process or data processing.

- **Use Case Diagram**
 - Shows interactions between external actors and the system.
- **Sequence Diagram**
 - Shows the sequence of interactions between actors and system components.
- **Class Diagram**
 - Shows object classes and the associations between them.
- **State Diagram**
 - Shows how the system responds to internal and external events by moving between states.

Use of Graphical Models

- **For discussion**
 - Used to facilitate discussions about existing or proposed systems.
 - Models may be incomplete or simplified.
 - **For documentation**
 - Used to document existing systems accurately, though not necessarily completely.
 - **For implementation**
 - Used as detailed system descriptions from which implementations may be generated.
 - Such models should be both correct and complete.
-

Context Models

- **Context models** are used to illustrate the **operational context** of a system.
- They show **what lies outside the system boundaries** and how the system relates to its environment.
- They help determine what functionality belongs inside the system and what is provided by external systems.

System Boundaries

- System boundaries define **what is inside** and **what is outside** the system.
- They show other systems that:
 - Interact with the system.
 - Depend on the system.
 - Are used by the system being developed.
- The position of the system boundary has a significant effect on system requirements.
- Deciding system boundaries may involve social, organizational, and political considerations.
- Different departments may try to increase or reduce their influence or workload by changing the system boundary.

Purpose of Context Models

- Clarify the environment in which the system operates.
- Identify external systems connected to the system.
- Help stakeholders understand system scope.
- Detect overlaps with existing systems and decide where new functionality should be implemented.
- Support early requirements analysis and reduce development cost and effort.

Architectural Models

- Context models are often represented using **architectural models**.
- These models show the system and its relationships with other external systems.

Process Perspective

- Context models show **which external systems exist**, but **do not show how the system is used** within business processes.
- To model business processes, **process models** are used.
- **UML Activity Diagrams** can be used to represent these business process models.

Mentcare System Example

- The **Mentcare System** interacts with several external systems in its environment, including:
 - Appointment system
 - General patient record system
 - Management reporting system
 - Hospital bed allocation system
 - Statistics/research system
 - The context model shows these connections and data-sharing relationships without describing their internal operations.
-

Interaction Models

- **Interaction models** describe how a system interacts with:
 - External users (actors).
 - Other systems.
 - Internal system components.
- They help in understanding communication between different parts of the system.
- **Use case diagrams** and **sequence diagrams** are commonly used for interaction modeling.

Importance of Interaction Models

- Help identify **user requirements** by modeling user interactions with the system.
- Show **system-to-system interactions**, highlighting possible communication issues.
- Show **component interactions**, helping assess whether the proposed design can meet performance and dependability requirements.

Use Case Modeling

- Use cases were originally developed to support **requirements elicitation** and are now part of UML.

- Each use case represents a **specific task or interaction** between an external actor and the system.
- Actors can be:
 - People (users).
 - Other software systems.
- Use cases provide:
 - A high-level graphical overview.
 - A detailed textual description of the interaction.
- A complete set of use cases should cover all major interactions with the system.

Sequence Diagrams

- Sequence diagrams are UML diagrams used to model interactions between actors and system objects.
- They show the **order (sequence)** in which messages or actions occur during a particular use case.
- Actors and objects are shown at the top of the diagram with vertical lifelines.
- Interactions are represented by arrows exchanged between these lifelines.
- They provide more detail than use case diagrams by illustrating the flow of events step by step.

Benefits of Interaction Models

- Improve understanding of how users and systems communicate.
- Help discover missing or incorrect requirements.
- Make it easier to analyze system behavior before implementation.
- Support the design and validation of software architecture.

UML Diagrams Used for Interaction Modeling

- **Use Case Diagram**
 - Shows interactions between external actors and the system.
 - Gives an overview of system functionality from the user's perspective.
- **Sequence Diagram**

- Shows the chronological sequence of interactions between actors and system components.
 - Describes message flow during a specific scenario or use case.
-

Structural Models

- **Structural models** show the **organization of a software system**, its components, and the relationships between those components.
- They help in understanding and designing the **system architecture**.
- Structural models can be:
 - **Static models** – show the structure of the system design.
 - **Dynamic models** – show the organization of the system while it is executing.

Purpose of Structural Models

- Represent the overall organization of a system.
- Show how different components are related.
- Help in discussing and designing the system architecture.
- Provide a clear view of the system's static structure.

Class Diagrams

- **Class diagrams** are the main UML diagrams used for structural modeling.
- They show:
 - The classes in a system.
 - The associations (relationships) between those classes.
- Used mainly when developing **object-oriented systems**.

Object Classes

- An **object class** is a general definition or blueprint for a type of object in the system.
- During early software development, classes often represent real-world entities such as:

- Patient
- Doctor
- Prescription
- Consultation

Associations

- An **association** is a relationship or link between two or more classes.
- It indicates that objects of one class are connected to or interact with objects of another class.

Generalization

- Generalization represents an **inheritance relationship** between classes.
- A more general class (superclass) can have one or more specialized classes (subclasses).
- Subclasses inherit the attributes and behavior of the superclass while adding their own specific features.

Benefits of Structural Models

- Provide a clear picture of system organization.
 - Help developers understand relationships among system components.
 - Support software architecture and object-oriented design.
 - Improve communication among designers and stakeholders.
-

Behavioural Models

- **Behavioral models** describe the **dynamic behavior** of a system while it is executing.
- They show how the system behaves or responds when it receives a stimulus from its environment.
- They help understand how the system processes data and reacts to events.

Types of Stimuli

- **Data-driven stimulus**

- Data arrives that must be processed by the system.
- The system performs operations based on the input data.
- **Event-driven stimulus**
 - An event triggers system processing.
 - Events may or may not have associated data.

Data-Driven Modeling

- Used mainly for business systems that process input data.
- The system's behavior is controlled primarily by incoming data.
- Data-driven models show:
 - The sequence of processing steps.
 - How input data is transformed into output.
- They are useful during requirements analysis because they illustrate end-to-end processing.

Activity Diagrams

- **UML Activity Diagrams** are commonly used for behavioral modeling.
- They represent the flow of activities or operations in a process.
- Each activity corresponds to one step in the processing sequence.
- They show:
 - The order of execution of activities.
 - Decision points and alternative paths.
 - The overall workflow of a process.

State Diagrams

- **State diagrams** model how a system responds to internal and external events.
- They show:
 - Different states of the system.
 - Transitions between states.
 - Events that trigger those transitions.

- Useful for systems whose behavior depends on their current state.

Benefits of Behavioral Models

- Describe how a system behaves during execution.
 - Help understand data processing and event handling.
 - Support requirements analysis and system design.
 - Make it easier to visualize workflows and dynamic system behavior.
 - Help identify missing or incorrect behavior before implementation.
-

Model-Driven Architecture

- **Model-Driven Architecture (MDA)** is a software development approach in which a system is represented as a set of models that can be automatically transformed into executable code.
- It is based on **Model-Driven Engineering (MDE)**, where models are the primary development artifacts instead of source code.
- The goal is to separate the system's functionality from platform-specific implementation details.

Basic Idea

- Create a high-level model of the system.
- Keep the model independent of implementation technology.
- Transform the model into one or more platform-specific models.
- Generate complete or partial system implementations from these models.

Advantages

- Promotes platform independence.
- Improves reuse of system models across different technologies.
- Makes it easier to generate implementations for multiple platforms.
- Helps maintain consistency between design and implementation.
- Reduces manual coding effort through automated code generation.

Limitations

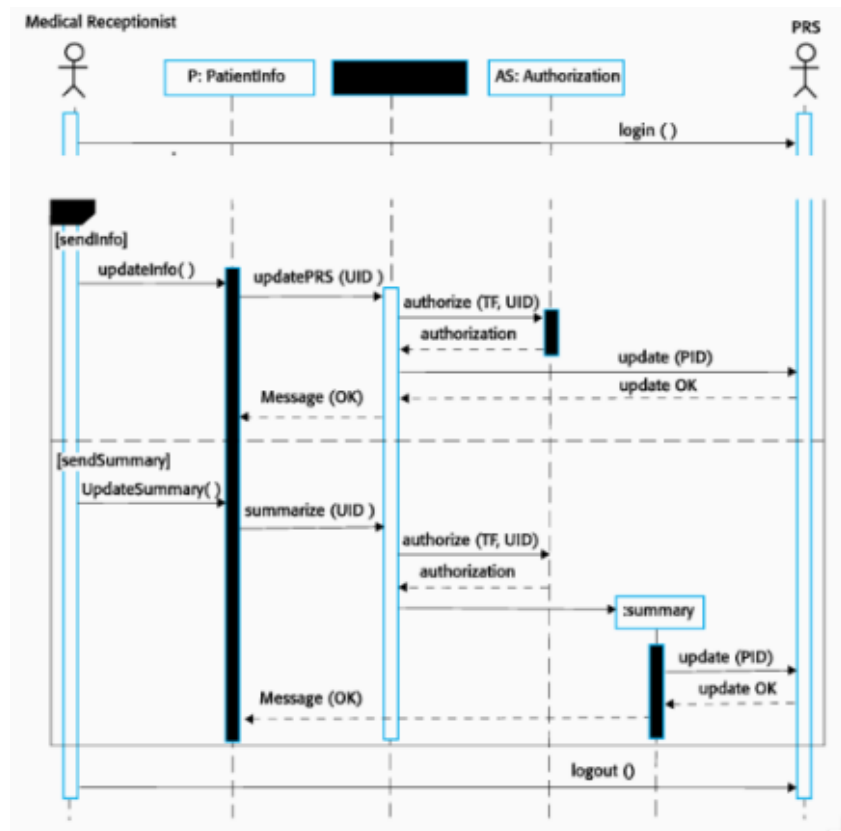
- Requires specialized tool support to transform models between different levels.
- Suitable tools may not always be available and may need customization.
- Organizations may hesitate to depend on specialized tools, especially for long-lived systems.
- The abstractions useful for design discussions may not be appropriate for direct implementation.
- For many complex systems, challenges such as requirements engineering, security, dependability, integration with legacy systems, and testing are often more significant than coding.
- The benefits of platform independence are greatest for large, long-life systems.
- For smaller software products and information systems, the cost of adopting MDA may outweigh its benefits.
- The popularity of agile development has reduced attention toward model-driven approaches.

Key Points

- MDA represents software as a collection of models rather than focusing directly on source code.
 - Models can be automatically transformed into executable implementations.
 - It emphasizes platform-independent design followed by platform-specific transformation.
 - It is most beneficial for large and long-term software systems where reuse and portability are important.
-

Update and Retrieve Transactions for Mentcare System

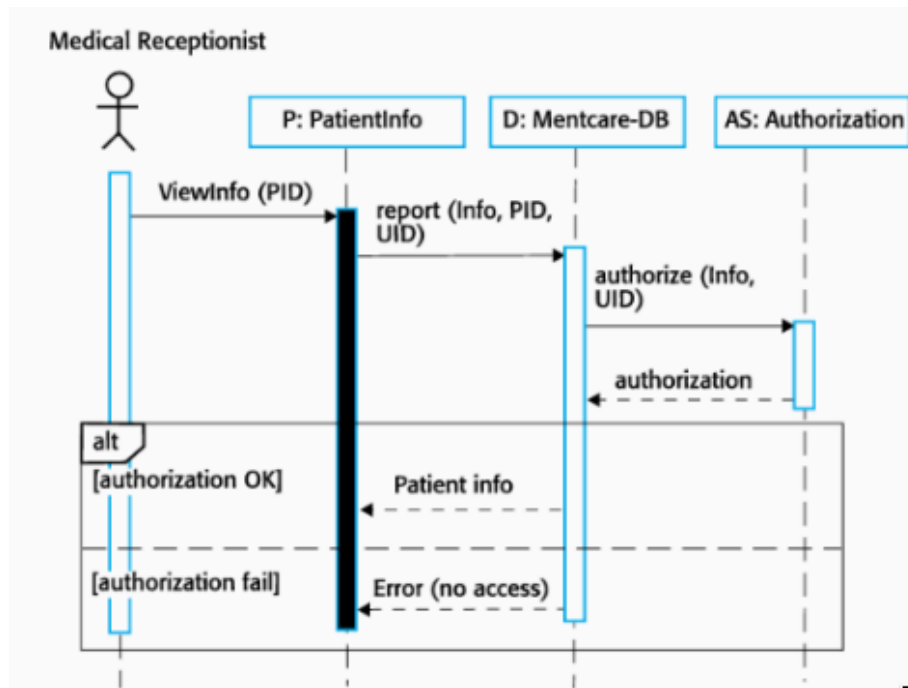
Update Transaction



Sequence:

- Medical Receptionist logs in.
- Sends `updateInfo()` to `PatientInfo`.
- `PatientInfo` sends `updatePRS(UID)` to `Mentcare-DB`.
- `Mentcare-DB` requests authorization using `authorize(TF, UID)`.
- Authorization system returns the result.
- `Mentcare-DB` updates the patient record in `PRS` using `update(PID)`.
- `PRS` returns `update OK`.
- `Mentcare-DB` sends `Message(OK)` to `PatientInfo`.
- `PatientInfo` confirms the update to the Medical Receptionist.

Retrieve Transaction



Sequence:

- Medical Receptionist requests `ViewInfo(PID)` .
- `PatientInfo` sends `report(info, PID, UID)` to `Mentcare-DB` .
- `Mentcare-DB` verifies access with the Authorization System.
- After successful authorization, the patient information is retrieved.
- `Mentcare-DB` sends the report to `PatientInfo` .
- `PatientInfo` displays the patient information to the Medical Receptionist.

Activity Diagram for Insulin Pump

- The **Insulin Pump Control System** is a medical embedded system that automatically monitors a diabetic patient's blood sugar level and delivers the required amount of insulin.
- Its main purpose is to maintain normal blood glucose levels without manual intervention.

Activity Diagram

- **Start**
 - The insulin pump system begins its operation.

- **Measure Blood Sugar**
 - A **microsensor** continuously measures a blood parameter related to the patient's sugar level.
- **Process Sensor Data**
 - The collected sensor readings are processed and analyzed by the controller.
- **Calculate Insulin Dose**
 - The controller calculates the correct amount of insulin required based on:
 - Current blood sugar level.
 - Previous insulin injections.
- **Generate Control Commands**
 - The system generates commands to operate the insulin pump according to the calculated dosage.
- **Deliver Insulin**
 - The pump injects the required amount of insulin into the patient's body.
- **Record Dosage Information**
 - The system records the insulin dose delivered for monitoring and safety purposes.
- **Repeat the Process**
 - The monitoring and delivery cycle continues continuously to maintain proper blood glucose levels.
- **End**
 - The process continues in a loop while the system is operational.

